

Pricing & Discount Engine

Tiered commission · service fees · founding programs · discount validation · Stripe integration · admin operations

Tato dokumentace popisuje pricing engine InkLink — co dělá, jak peníze tečou, a jak ho operuješ. Verze odpovídá kódu v **pricing/** module a feature flagu **USE_NEW_PRICING_ENGINE**. Vygenerováno z aktuálního config (pricing/config.py).

1. Big picture — co dělá co

Pricing engine je pure-function modul mimo **server.py**, který za jeden volání vypočítá kompletní ekonomiku bookingu: kolik klient zaplatí, kolik dostane tatér, kolik nám zůstane, kolik si vezme Stripe, jaká sleva se aplikuje.

Booking flow s engine zapnutým (USE_NEW_PRICING_ENGINE=1):

1. Klient otevře sketch a klikne Rezervovat
2. Frontend volitelně volá **POST /api/discounts/preview** aby zobrazil náhled slevy
3. Frontend volá **POST /api/bookings** s **discount_code**
4. Backend volá **pricing.calculate_booking_economics()** — pure function:
 - tier_for_price(gross) → 12 % / 8 % / 5 % podle ceny
 - commission_for_artist(gross, founding_status)
 - service_fee_for(gross, client_founding)
 - stripe_fee_for(client_pays_total)
5. Backend uloží snapshot do **economics_snapshots** (immutable ledger)
6. Backend zapíše do **discount_redemptions** (audit)
7. Emit telemetry event **booking.economics_calculated**
8. Klient zaplatí přes Stripe → webhook potvrdí → status confirmed
9. Po tetování → status completed → emit **booking.completed**
10. Refund/dispute → další snapshot s kind='refund' nebo 'adjust'

2. Architektura kódu

Business rules žijí v **pricing/** module, ne v `server.py`. Změna rate znamená editaci jednoho souboru (`pricing/config.py`); `server.py` se nemění.

Soubor	Co dělá
<code>pricing/config.py</code>	Single source of truth pro rates (tiers, floor, fees, windows).
<code>pricing/economics.py</code>	<code>calculate_booking_economics()</code> pure function. Decimal math.
<code>pricing/discounts.py</code>	<code>validate_discount()</code> s error codes (<code>DISCOUNT_TOO_LARGE...</code>).
<code>pricing/telemetry.py</code>	<code>emit_event(name, payload, conn)</code> → <code>telemetry_events</code> table.
<code>pricing/admin.py</code>	<code>admin_kpis()</code> — agreguje <code>economics_snapshots</code> .
<code>tests/test_pricing.py</code>	41 unit testů. Tier boundaries, founding days, discount cap.
<code>server.py</code>	Integration: volá engine, ukládá snapshots, webhooks, endpoints.
<code>public/admin.html</code>	UI: KPI dashboard, promo CRUD, telemetry feed.

Klíčový princip: `pricing/` je dependency-free (jen `Decimal/datetime` z `stdlib`). Pure functions znamená že same input → same output, žádný side effect, ideální k unit testování. Idempotency: zavoláš dvakrát → stejný výsledek.

3. Money flow — kdo komu kolik

Příklad: tetování za 5 000 Kč od founding artista (day 35) pro non-founding klienta.

Krok	Kč	Komu / kam
Klient platí (gross + service fee)	5 150	→ Stripe
Stripe fee (1.5 % + 6 Kč)	-76	cost
Funds na platform balance	5 074	↓
Artist commission (5 % flat — founding flat window)	-250	InkLink keeps
Artist payout (gross – commission)	4 750	→ tatér (transfer)
InkLink net (service_fee + commission – stripe_fee)	324	naše net revenue

Inviolable rules (vynutí engine):

1. Artist payout = gross – commission. Discount **NIKDY** nezasáhne artistu.
2. Discount ≤ 60 % naší komise.
3. Net revenue nikdy záporné (kromě founding artist days 1–30 — explicit choice).

4. Aktuální rates (pricing/config.py)

Tiered commission:

Gross cena	Commission rate
0–2999 Kč	12 %
3000–9999 Kč	8 %
10000+ Kč	5 %

Minimum commission floor: 200 Kč. Pokud tiered % dá méně, vezmeme floor. Founding artist days 1–30 floor neaplikuje (intentional).

Service fee (od klienta):

- Sazba: **3 %** z gross
- Cap: **150 Kč**
- Founding client (prvních 500 signups): **waived navždy**

Founding artist program:

- Day 1–30: **0 %** commission (i pod floor)
- Day 31–90: **flat 5 %** bez floor
- Day 91+: standard tiered
- Clock start: první completed booking nastaví founding_artist_started_at

Stripe fees:

Card type	%	Fix Kč
card_eea	1.5 %	6
card_non_eea	3.4 %	6

+ **1 %** surcharge pokud karta není CZK.

5. Discount engine

Ti typy slev:

Typ	Amount	Eligibility	Kde se konfiguruje
WELCOME	200 Kč	First booking only	Code path v engine
REFERRAL	300 Kč	Po referral linku	Code path + referrals table
MANUAL_PROMO	Variabilní	Admin issuance	discount_codes table

Cap rule

Discount ≤ 60 % komise. Příklady:

- Booking 1 500 Kč → commission = 200 (floor). Max sleva = 120. **WELCOME 200 Kč REJECT.**
- Booking 5 000 Kč → commission = 400. Max sleva = 240. **WELCOME 200 Kč OK.**

Error codes (stable, surface in admin alerts)

DISCOUNT_TOO_LARGE · DISCOUNT_NOT_ELIGIBLE · DISCOUNT_ALREADY_USED ·
DISCOUNT_STACKING · NEGATIVE_PAYOUT · NEGATIVE_NET

6. Stripe integration

Architektura: destination charges (souhlasné, nemigrovali jsme). Funds tebou rovnou tatérovi na jeho Connect account; platform si vezme application_fee_amount.

PaymentIntent.create() ukládá metadata.inklink_booking_id aby webhook našel booking.

Webhook handlers (idempotent):

Event	Akce
payment_intent.succeeded	booking → confirmed, emit booking.payment_succeeded
payment_intent.payment_failed	booking → payment_failed
charge.refunded	new snapshot kind=refund, booking → refunded
charge.dispute.created	booking → disputed (admin řeší ručně)
account.updated	sync stripe_charges_enabled / payouts_enabled artisty

Idempotency

Stripe retries webhooks aggressively. Před processing INSERT event_id do **processed_stripe_events**. Pokud UNIQUE fails → 200 OK + skip. Žádný duplicate accounting možný.

7. Telemetry & audit

Každá business událost se zapíše do tabulky **telemetry_events** jako JSON. Admin vidí v dashboardu (/admin → Telemetry events sekce).

Event name	Kdy se emituje
booking.economics_calculated	Při PaymentIntent creation s full snapshotem
booking.payment_succeeded	Webhook payment_intent.succeeded
booking.payment_failed	Webhook payment_intent.payment_failed
booking.completed	Po oboustranném potvrzení dokončení
booking.refunded	Webhook charge.refunded
booking.disputed	Webhook charge.dispute.created
discount.applied	Sleva úspěšně aplikována na booking
discount.rejected	Sleva neprošla validací (s error_code)
founding_artist.clock_started	První completed booking → start 30/90 day window
reconciliation.completed	Daily diff Stripe vs internal (>10 Kč = warning)

8. Admin operations — co dleáš ruřit

Zapnutí engine v produkci

V Railway dashboardu nastav env proměnnou:

```
USE_NEW_PRICING_ENGINE=1
```

Flask běže env per-request, žádný restart není potřeba. Existing bookings beze změny — flag ovlivňuje jen nové.

Vytvoření MANUAL_PROMO kódu

V /admin → sekce "Discount kódy" vyplíš code (např. SPRING25), amount, volitelně max_uses a expirace. Klikni "+ Vytvořit". Klient pak při bookingu zadá kód, engine validuje cap a aplikuje.

Enroll founding artist

Najdi user id (např. v admin sekce) a v DB:

```
UPDATE users SET founding_artist=1 WHERE id IN (1,2,3,...);
```

founding_artist_started_at se nastaví automaticky při jejich prvním completed booking — žádný manuál nutný.

Daily reconciliation cron

V Railway nastav env RECONCILE_TOKEN=<random> a cron schedule:

```
0 6 * * * curl "https://www.inklink.club/api/cron/reconcile?token=$RECONCILE_TOKEN"
```

Porovná internal total vs Stripe balance. Diff > 10 Kč zaloguje warning v telemetry events. Pomáhá najít bugs dřív než se nakumulují.

KPI dashboard

/admin → sekce "Pricing engine — KPIs". Date range + tier filter. Default posledních 30 dní. Vidíš GMV, net revenue, take rate, discount cost, refund loss, Stripe cost, avg booking.

9. Co se NESTALO (zámrn) + rollback

Zámrné non-goals (founder rozhodnutí):

- **Žádná migrace na separated charges** — funds dál teou rovnou tatérovi přes destination charges. Real escrow s delayed transfers = budoucí migrace.
- **Žádný 24h cooldown** před payout — destination charges to nemají.
- **Žádný native push pro disputes** — admin emaily zatím chybí.

Rollback v produkci

Pokud něco bouchne, okamžitě:

```
USE_NEW_PRICING_ENGINE=0
```

Engine se přestane volat, existing booking flow běží na flat 8 %. DB schema (snapshots, founding flags, discount tables) zůstává — neškodí.

Fail-safe v kódu

POST /api/bookings má try/except kolem volání engine. Pokud engine throws, log warning **[pricing-engine] error, falling back to legacy** a booking se vytvoří se starým flat 8 %. Klient nic nepozná. Idempotency table chrání před duplicate processing při Stripe retry.

Validation status

- **41 unit test** všechny passují (tests/test_pricing.py)
- Pure function determinism + idempotency verified
- Tier boundaries, founding day transitions, discount cap edge cases covered